

Heaven's light is our guide"

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

Compiler Design

Course No. : 701

Chapter 3: Lexical Analysis

Prepared By : Julia Rahman

3.1 The Role of the Lexical Analyzer

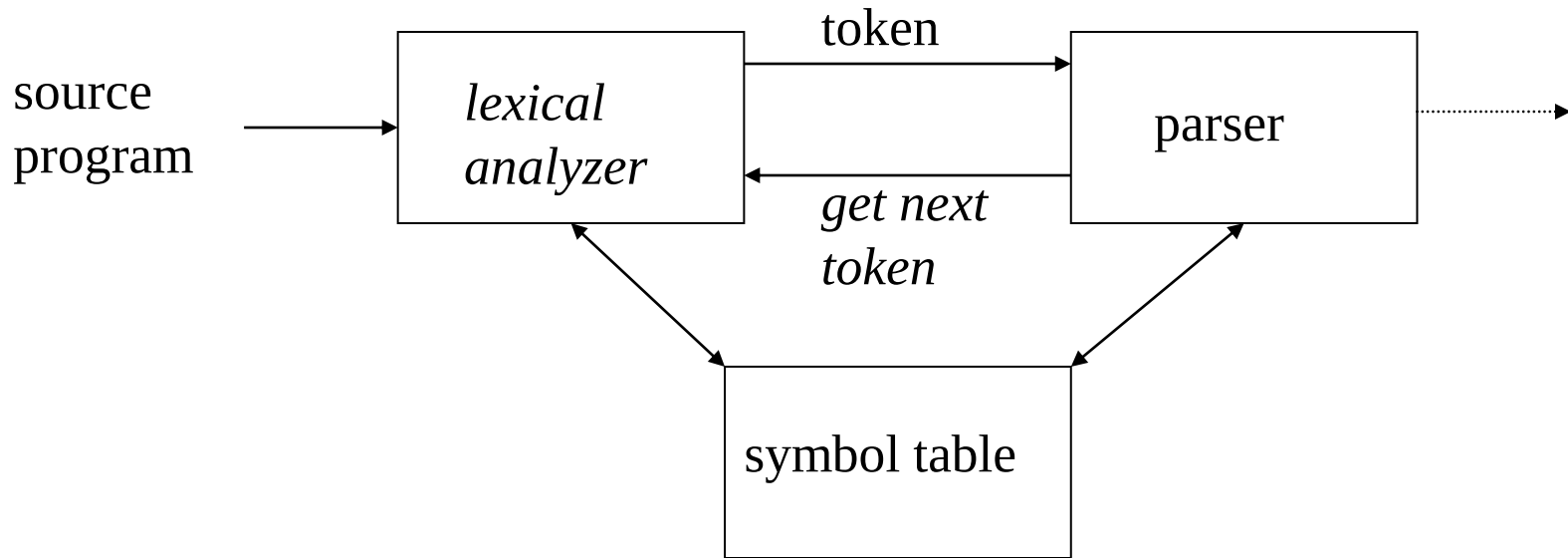


Figure 3.1: Interaction of lexical analyzer with parser

3.1 The Role of the Lexical Analyzer

- ✓ First phase of a compiler
- ✓ Main task
 - To read the input characters
 - To produce a sequence of tokens used by the parser for syntax analysis
 - As an assistant of parser
- ✓ Interaction of lexical analyzer with parser
- ✓ Processes in lexical analyzers
 - Scanning
 - Pre-processing
 - Strip out comments and white space
 - Macro functions
 - Correlating error messages from compiler with source program
 - A line number can be associated with an error message
- ✓ Lexical analyzer are divided into a cascade of two phases:
 - a) **Scanning** consists of the simple processes that do not require tokenization of the input, such as deletion of comments and compaction of consecutive whitespace characters into one.
 - b) **Lexical analysis** proper is the more complex portion, where the scanner produces the sequence of tokens as output.

3.1 The Role of the Lexical Analyzer

Why the analysis portion of a compiler is normally separated into lexical analysis and parsing (syntax analysis) phases?

- 1) Simplicity of design.
- 2) Compiler efficiency is improved
- 3) Compiler portability is enhanced.

Tokens, Patterns and Lexemes

Token:

- ✓ A classification for a common set of strings or a group of characters having a collective meaning
- ✓ Examples: Integer, Float, a particular keyword, or a sequence of input characters denoting an identifier

Pattern:

- ✓ A description of the form that the lexemes of a token may take
- ✓ The rule describing how a token can be formed

Lexeme:

- ✓ A lexeme is a sequence of characters in the source program that matches the pattern for a token
- ✓ Actual sequence of characters that matches pattern and is classified by a token
- ✓ Identifiers: x, count, name, etc. Integers: 345, 20 -12, etc.

3.1 The Role of the Lexical Analyzer



Example: C statement

```
printf( "Total = %d\n" , score ) ;
```

both ***printf*** and ***score*** are lexemes matching the pattern for token id, and " Total = %d\n" is a lexeme matching literal.

Token	Sample Lexemes	Informal Description of Pattern
const	const	const
if	if	characters i, f
relation	<, <=, =, < >, >, >=	< or <= or = or < > or >= or >
id	pi, <u>count</u> , <u>D2</u>	letter followed by letters and digits
<u>num</u>	<u>3.1416</u> , 0, <u>6.02E23</u>	any numeric constant
literal	"core dumped"	any characters between " and " except"

Classifies
Pattern

Actual values are critical. Info is :

1. Stored in symbol table
2. Returned to parser

3.1 The Role of the Lexical Analyzer

Lexical Errors:

- ✓ Some errors are out of power of lexical analyzer to recognize:
fi (a == f(x)) ...
- ✓ However it may be able to recognize errors like:
d = 2r
- ✓ Such errors are recognized when no pattern for tokens matches a character sequence
- ✓ Error Handling is very localized, with Respect to Input Source
For example: while (x := 0) do
generates no lexical errors in PASCAL
- ✓ **Error recovery:**
 - Deleting an extraneous character
 - Inserting a missing character
 - Replacing an incorrect character by a correct character
 - Transposing two adjacent characters(such as , fi→if)

3.2 Input Buffering

- ✓ Sometimes lexical analyzer needs to look ahead some symbols to decide about the token to return
 - In C language: we need to look after -, = or < to decide what token to return
 - In Fortran: DO 5 I = 1.25
- ✓ We need to introduce a two buffer scheme to handle large look-aheads safely
- ✓ Two-buffer input scheme to look ahead on the input and identify tokens

1) Buffer pairs

- ✓ Because of the amount of time taken to process characters and the large number of characters that must be processed during the compilation of a large source program, specialized buffering techniques have been developed to reduce the amount of overhead required to process a single input character.
- ✓ Two pointers to the input are maintained:
 1. Pointer lexemeBegin: marks the beginning of the current lexeme,

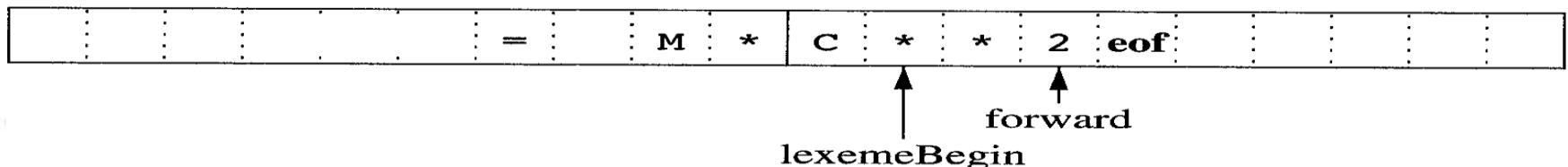


Figure 3.3: Using a pair of input buffers

3.2 Input Buffering

2. Pointer forward: scans ahead until a pattern match is found

2) Sentinels(Guards)

- ✓ For each character read, we make two tests: one for the end of the buffer, and one to determine what character is read.
- ✓ The sentinel is a special character that cannot be part of the source program, and a natural choice is the character *eof*.
- ✓ Note that *eof* retains its use as a marker for the end of the entire input.
- ✓ Any *eof* that appears other than at the end of a buffer means that the input is at an end.

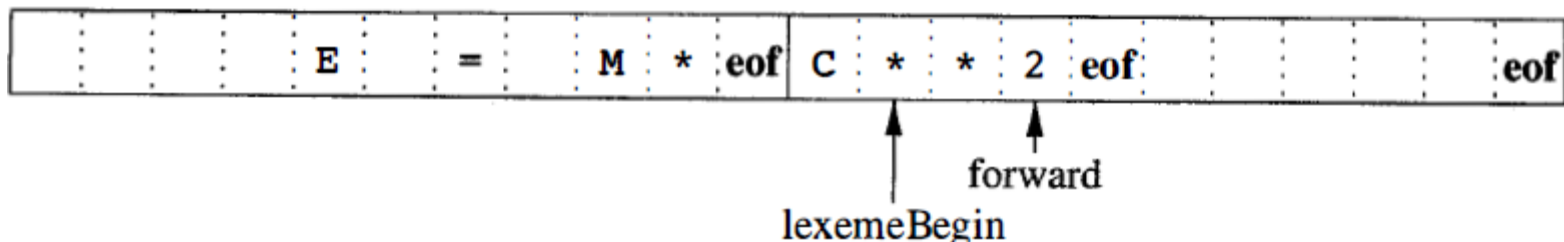


Figure 3.4: Sentinels at the end of each buffer

3.3 Specification of Tokens

+ Alphabet or character:

- ✓ Finite set of symbols
- ✓ Example $\{0,1\}$, or $\{a,b,c\}$, or $\{n,m, \dots, z\}$

+ String:

- ✓ Finite sequence of symbols from an alphabet.
- ✓ Example 0011 or abbca or AABBC ...
- ✓ If S is a string, then $|S|$ is the **length** of S , i.e. the **number of symbols** in the string S .
- ✓ ϵ : **Empty String**, with $|\epsilon| = 0$

+ Language:

A set of strings over an alphabet.

+ Language Concepts:

- ✓ A language, L , is simply any set of strings over a fixed alphabet.

Alphabet

$\{0,1\}$

$\{a,b,c\}$

$\{A, \dots, Z\}$

$\{A, \dots, Z, a, \dots, z, 0, \dots, 9, +, -, \dots, <, >, \dots\}$

Languages

$\{0,10,100,1000,100000\dots\}$

$\{abc,aabbcc,aaabbbccc,\dots\}$

$\{TEE,FORE,BALL,\dots\}$

$\{ \text{All legal PASCAL progs} \}$

$\{ \text{All grammatically correct English sentences} \}$

3.3 Specification of Tokens

+ Operation on languages (a set):

- ✓ **union** of L and M, $L \cup M = \{s | s \text{ is in } L \text{ or } s \text{ is in } M\}$
- ✓ **concatenation** of L and M
 $LM = \{st | s \text{ is in } L \text{ and } t \text{ is in } M\}$
- ✓ Kleene closure of L, $L^* = \bigcup_{i=0}^{\infty} L^i$
- ✓ Positive closure of L, $L^+ = \bigcup_{i=1}^{\infty} L^i$
- ✓ Example: $L = \{aa, bb, cc\}$, $M = \{abc\}$

+ Example:

$$L = \{A, B, C, D\} \quad D = \{1, 2, 3\}$$

$$L \cup D = \{A, B, C, D, 1, 2, 3\}$$

$$LD = \{A1, A2, A3, B1, B2, B3, C1, C2, C3, D1, D2, D3\}$$

$$L^2 = \{AA, AB, AC, AD, BA, BB, BC, BD, CA, \dots DD\}$$

$$L^4 = L^2 L^2 = ??$$

$$L^* = \{ \text{All possible strings of } L \text{ plus } \epsilon \}$$

$$L^+ = L^* - \epsilon$$

$$L(L \cup D) = ??$$

$$L(L \cup D)^* = ??$$

3.3 Specification of Tokens

Example:

- ✓ Suppose: S is the string banana

Prefix : removing zero or more trailing symbols of string - **ban**, **banana**

Suffix : deleting zero or more leading symbols of string - **ana**, **banana**

Substring : deleting prefix or suffix from string - **nan**, **ban**, **ana**, **banana**

Subsequence: deleting zero or more not necessarily contiguous symbols - **bnan**, **nn**

- ✓ Proper prefix, suffix, or substring cannot be all of S

- ❖ **Special Languages:** $\emptyset$ - EMPTY LANGUAGE
 ϵ - contains ϵ string only

3.3 Specification of Tokens

✚ Regular Expression:

- ✓ A notation that allows us to define a pattern in a high level language.
- ✓ a set of Rules / Techniques for constructing sequences of symbols (Strings) from an Alphabet.
- ✓ The rule of regular expression over alphabet
 - (1) ϵ is a regular expression that denote $\{\epsilon\}$, the set that contains the empty string.
 - (2) For each $a \in \Sigma$, a is a regular expression denote $\{a\}$, the set containing the string a .
 - (3) r and s are regular expressions denoting language $L(r)$ and $L(s)$. Then
 - a) $(r) | (s)$ is a regular expression denoting $L(r) \cup L(s)$
 - b) $(r)(s)$ is a regular expression denoting $L(r)L(s)$
 - c) $(r)^*$ is a regular expression denoting $(L(r))^*$
 - d) (r) is a regular expression denoting $L(r)$
- ❖ Last rule says that add additional pairs of parentheses around expressions without changing the language they denote.

3.3 Specification of Tokens

+ Regular language:

Each regular expression r denotes a language $L(r)$ (the set of sentences relating to the regular expression r)

+ Regular set:

A language denoted by a regular expression is said to be regular set.

+ Unnecessary pairs of parentheses can be avoided in regular expression if we adopt the conventions that :

- The unary operator $*$ has highest precedence and is left associative.
- Concatenation has second highest precedence and is left associative.
- $|$ has lowest precedence and is left associative.

3.3 Specification of Tokens

Example 3.3 :

Let $\Sigma = \{a, b\}$.

1. The regular expression $(a \mid b)$ denotes the set $\{a, b\}$.
2. $(a \mid b) (a \mid b)$ denotes $\{aa, ab, ba, bb\}$, the language of all strings of length two over the alphabet Σ . Another regular expression for the same language is $aa \mid ab \mid ba \mid bb$.
3. a^* denotes the language consisting of all strings of zero or more a's, that is, $\{\epsilon, a, aa, aaa, \dots\}$.
4. $(a \mid b)^*$ denotes the set of all strings consisting of zero or more instances of a or b, that is, all strings of a's and b's: $\{\epsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$. Another regular expression for the same language is $(a^* b^*)^*$.
5. $a \mid a^* b$ denotes the language $\{a, b, ab, aab, aaab, \dots\}$, that is, the string a and all strings consisting of zero or more a's and ending in b.

3.3 Specification of Tokens

AXIOM	DESCRIPTION
$r \mid s = s \mid r$	$\mid$ is commutative
$r \mid (s \mid t) = (r \mid s) \mid t$	$\mid$ is associative
$(r \ s) t = r (s \ t)$	concatenation is associative
$r (s \mid t) = r s \mid r t$ $(s \mid t) r = s r \mid t r$	concatenation distributes over $\mid$
$\epsilon r = r$ $r \epsilon = r$	ϵ Is the identity element for concatenation
$r^* = (r \mid \epsilon)^*$	relation between $*$ and ϵ
$r^{**} = r^*$	$*$ is idempotent

Figure 3.9: Algebraic laws for regular expressions

3.3 Specification of Tokens



Regular definition:

- ✓ gives names to regular expressions to construct more complicate regular expressions.
- ✓ If Σ is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form:

$$d_1 \rightarrow r_1$$

$$d_2 \rightarrow r_2$$

...

$$d_n \rightarrow r_n$$

where:

1. Each d_i is a new symbol, not in Σ and not the same as any other of the d's, and
2. Each r_i is a regular expression over the alphabet $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$.

3.3 Specification of Tokens

✚ **Example 3.4 :** C identifiers are strings of letters, digits, and underscores. Here is a regular definition for the language of C identifiers. We shall conventionally use italics for the symbols defined in regular definitions.

letter $\rightarrow A \mid B \mid C \mid \dots \mid Z \mid a \mid b \mid \dots \mid z$

digit $\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

identifier $\rightarrow \text{letter} (\text{letter} \mid \text{digit})^*$

✚ **Example 3.5 :** Unsigned numbers (integer or floating point) in Pascal are strings such as 5280, 39.37, 6.336E4, or 1.89E-4. The following regular definition provides a precise specification for this class of strings:

digit $\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

digits $\rightarrow \text{digit} \text{ digit}^*$

optional_fraction $\rightarrow .\text{digits} \mid \epsilon$

optional_exponent $\rightarrow (E(+ \mid - \mid \epsilon) \text{ digits}) \mid \epsilon$

num $\rightarrow \text{digits} \text{ optional_fraction} \text{ optional_exponent}$

3.3 Specification of Tokens



Shorthand Notation:

1) One or more instances:

- ✓ The unary, postfix operator $+$ means “one or more instances”.
- ✓ If r is a regular expression, then $(r)^+$ denotes the language $(L(r))^+.$
- ✓ The operator $+$ has the same precedence and associativity as the operator $*$.
- ✓ Two useful algebraic laws,

$$r^* = r^+ | \epsilon \text{ and}$$

$$r^+ = rr^* = r^*r$$

2) Zero or one instance:

- ✓ The unary postfix operator $?$ means "zero or one occurrence."
- ✓ $r?$ is equivalent to $r | \epsilon$, or put another way, $L(r?) = L(r) \cup \{\epsilon\}.$
- ✓ The $?$ operator has the same precedence and associativity as $*$ and $+$.

3) Character classes:

- ✓ set range of characters (replaces “|”)

$$[A-Z] = A | B | C | \dots | Z$$

$$[a-z] \text{ denotes } a|b|c|\dots|z$$

$$[A-Za-z] [A-Za-z0-9]$$

3.3 Specification of Tokens

 **Example:** Using these shorthands, we can rewrite the regular definition of Example 3.4 as:

letter_ $\rightarrow$ [A-Za-z_]

digit $\rightarrow$ [0-9]

id $\rightarrow$ letter_ (letter | digit) *

The regular definition of Example 3.5 can also be simplified:

digit $\rightarrow$ [0-9]

digits $\rightarrow$ digit+

optional_fraction $\rightarrow$ (.digits)?

optional_exponent $\rightarrow$ (E(+ | -) ? digits) ?

num $\rightarrow$ digits optional_fraction optional_exponent

3.3 Specification of Tokens

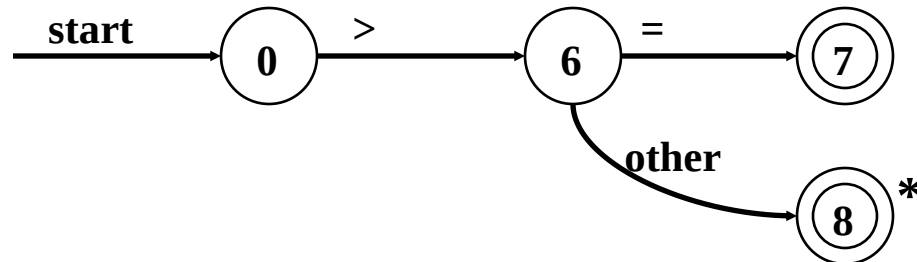


Transition Diagram:

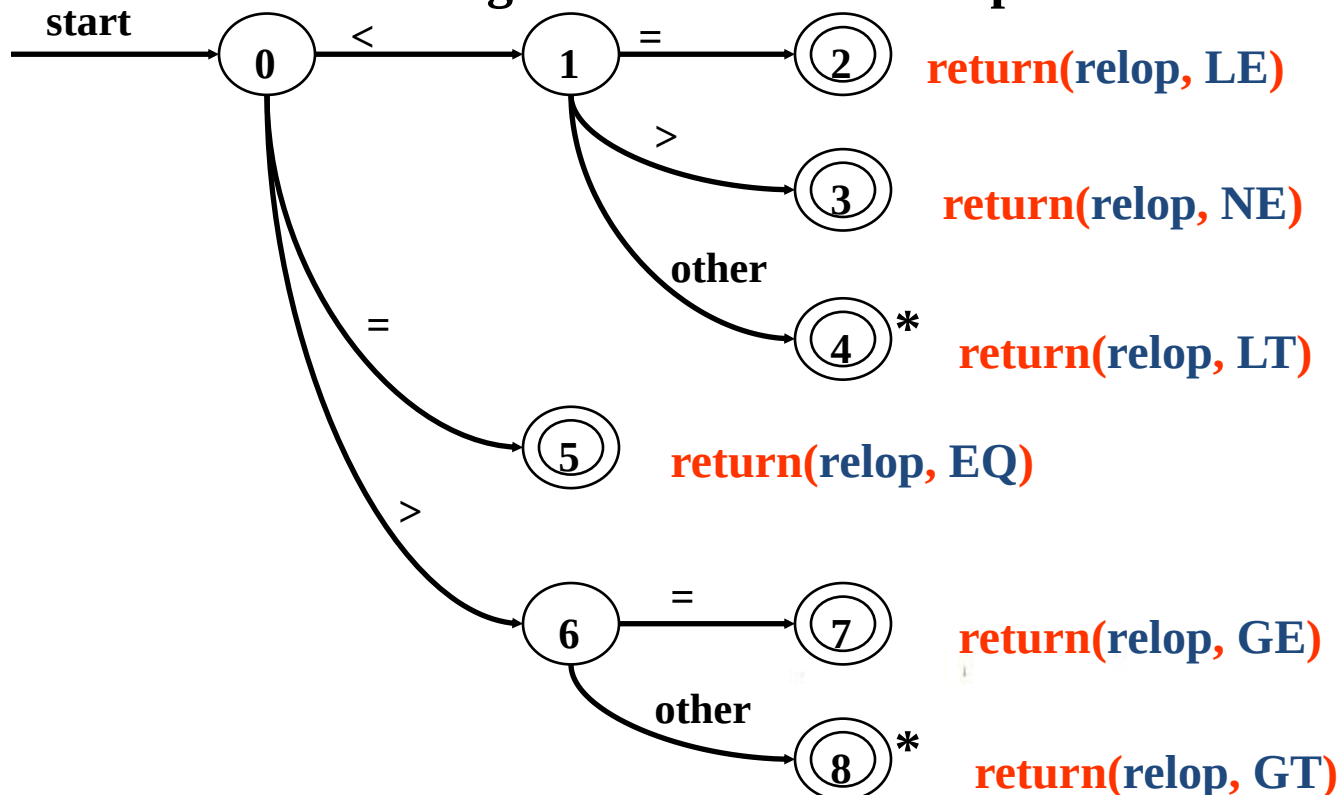
- ✓ Depict the actions that take place when a lexical analyzer is called by the parser to get the next token.
- ✓ Transition Diagrams (TD) are used to represent the tokens – these are automata!
- ✓ As characters are read, the relevant Transition Diagrams are used to attempt to match lexeme to a pattern
- ✓ Each Transition Diagrams has:
 - **States** : Represented by **Circles**
 - **Actions** : Represented by **Arrows** between states
 - **Start State** : Beginning of a pattern (**Arrowhead**)
 - **Final State(s)** : End of pattern (**Concentric Circles**)
- ✓ Each Transition Diagrams is **Deterministic** - No need to choose between 2 different actions !

3.3 Specification of Tokens

Transition Diagram for $\geq$:

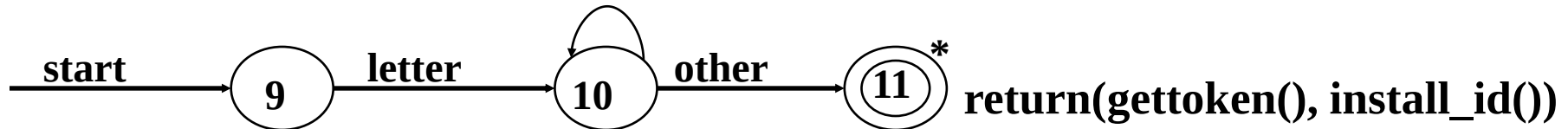


Example 3.7: Transition Diagram for Relational Operators:

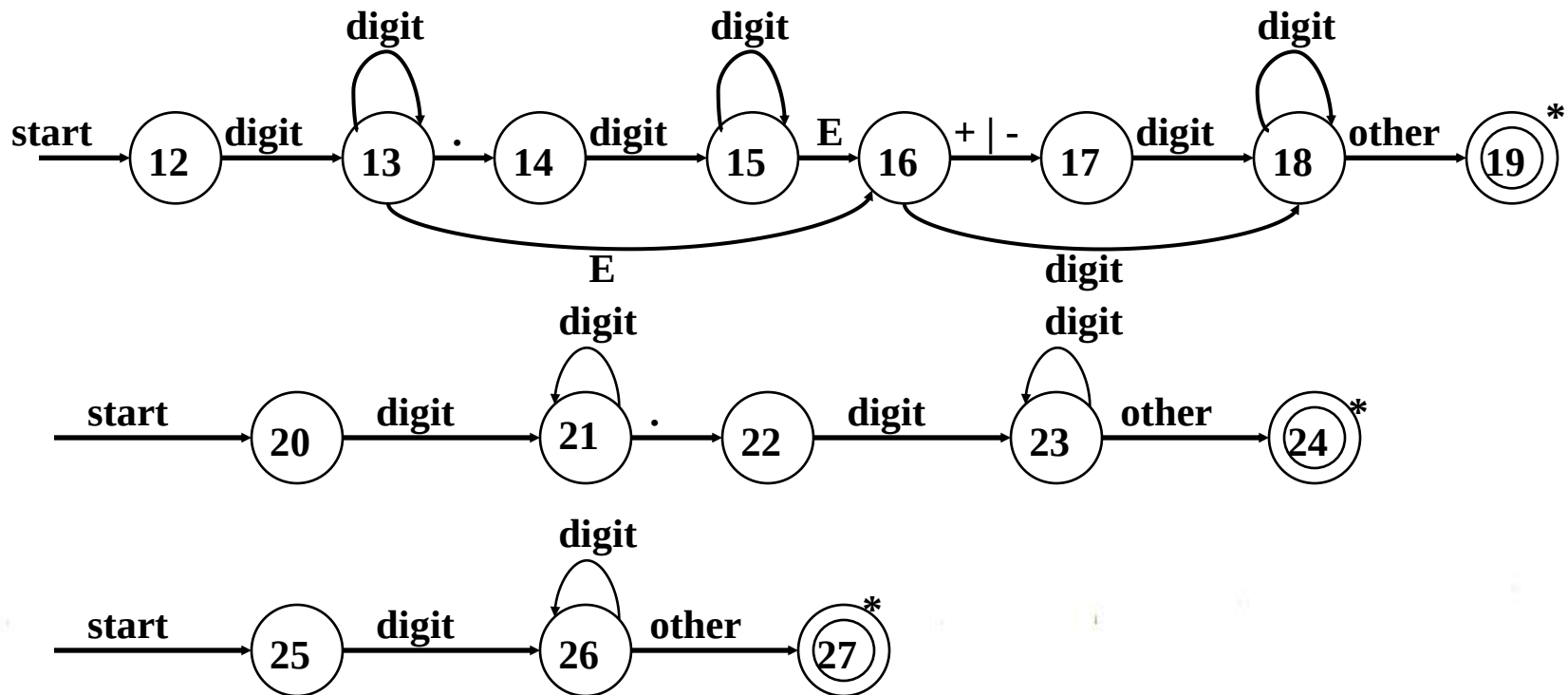


3.3 Specification of Tokens

Example 3.8: Transition Diagram for identifiers and keywords:
letter or digit



Transition Diagram for unsigned numbers in Pascal:



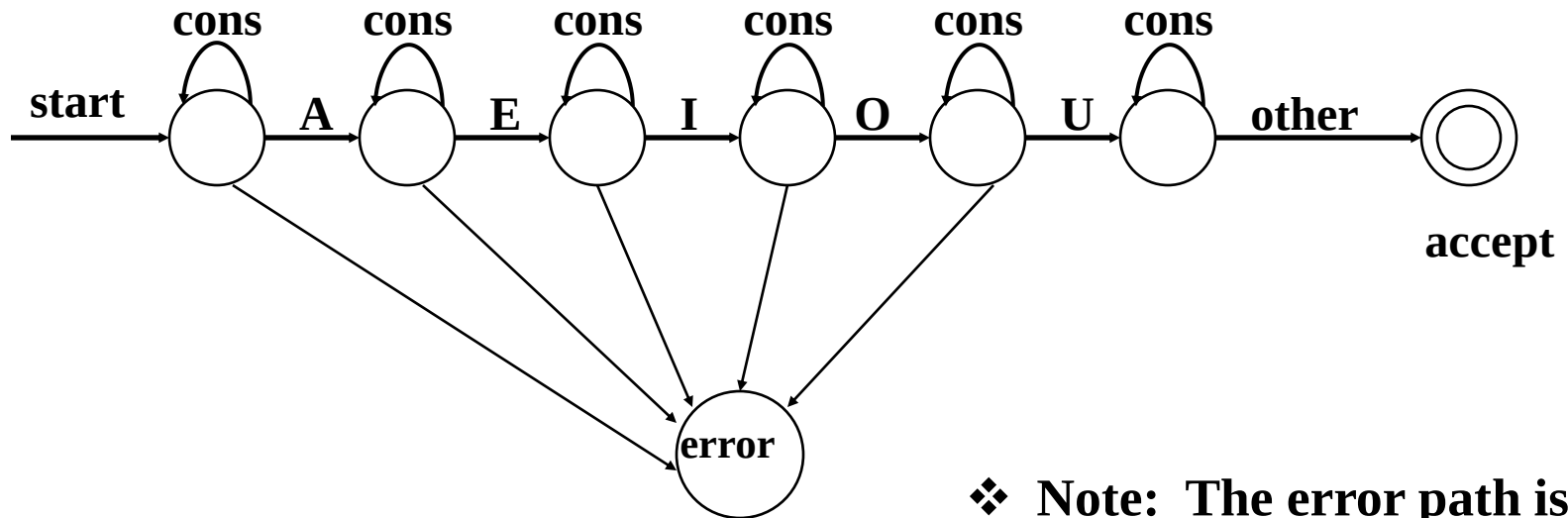
3.3 Specification of Tokens



What would the transition diagram (TD) for strings containing each vowel, in their strict lexicographical order, look like ?

Answer: $\text{cons} \rightarrow B \mid C \mid D \mid F \mid \dots \mid Z$

$\text{string} \rightarrow \text{cons}^* A \text{cons}^* E \text{cons}^* I \text{cons}^* O \text{cons}^* U \text{cons}^*$



❖ **Note:** The error path is taken if the character is other than a cons or the vowel in the lex order.

3.6 Finite Automata



Finite Automata:

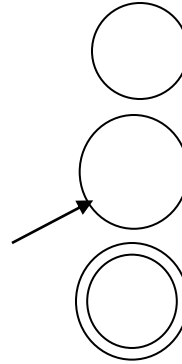
- ✓ Finite automata are recognizers; they simply say "yes" or "no" about each possible input string. OR
- ✓ A recognizer that takes an input string and determines whether it's a valid string of the language.
- ✓ A generalized transition diagram.
- ✓ Regular expressions is specification and Finite automata is implementation
- ✓ A finite automaton consists of:
 - An input alphabet Σ
 - A set of states S
 - A start state n
 - A set of accepting states $F \subseteq S$
 - A set of transitions $\text{state} \rightarrow \text{input state}$

3.6 Finite Automata

✓ Transition $s_1 \xrightarrow{a} s_2$

Is read, In state s_1 on input “a” go to state s_2

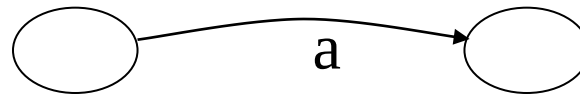
- A state



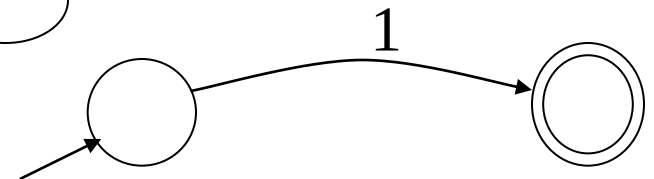
- The start state

- An accepting state

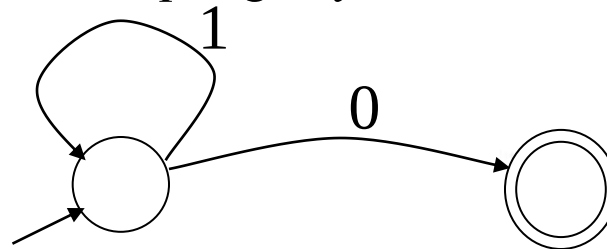
- A transition



- A finite automaton that accepts only “1”

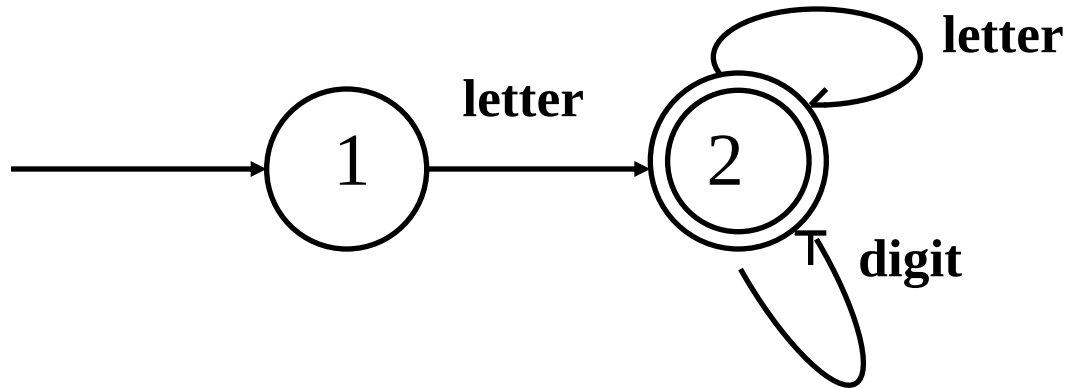


- A finite automaton accepting any number of 1's followed by a single 0,
Alphabet: {0,1}



3.6 Finite Automata

✚ The Finite Automata simulator for Identifiers is:



Which represent the rule: **identifier=letter(letter|digit)***

✚ Usage of Finite Automata:

- ✓ Precisely recognize the regular sets
- ✓ A regular set is a set of sentences relating to the regular expression

3.6 Finite Automata

 **Finite automata come in two flavors:**

a) Nondeterministic finite automata (NFA):

- ✓ Have no restrictions on the labels of their edges.
- ✓ Can have multiple transitions for one input in a given state
- ✓ Can have ϵ –moves
- ✓ Non-Deterministic Finite Automata (NFAs) easily represent regular expression, but are somewhat less precise

b) Deterministic finite automata (DFA):

- ✓ One transition per input per state
- ✓ No ϵ –moves
- ✓ Completely determined by input
- ✓ A DFA accepts an input string x if and only if there is some path in the transition graph from start state to some accepting state
- ✓ Deterministic Finite Automata (DFAs) require more complexity to represent regular expressions, but offer more precision.

3.6 Finite Automata

❖ An **NFA** is a mathematical model that consists of :

- ✓ S , a set of **states**
- ✓ Σ , the symbols of the **input alphabet**
- ✓ *move*, a **transition function**.
 - $\text{move}(\text{state}, \text{symbol}) \rightarrow \text{state}$
 - $\text{move} : S \times \Sigma \rightarrow S$
- ✓ A state, $s_0 \in S$, the **start state**
- ✓ $F \subseteq S$, a set of **final** or **accepting states**.

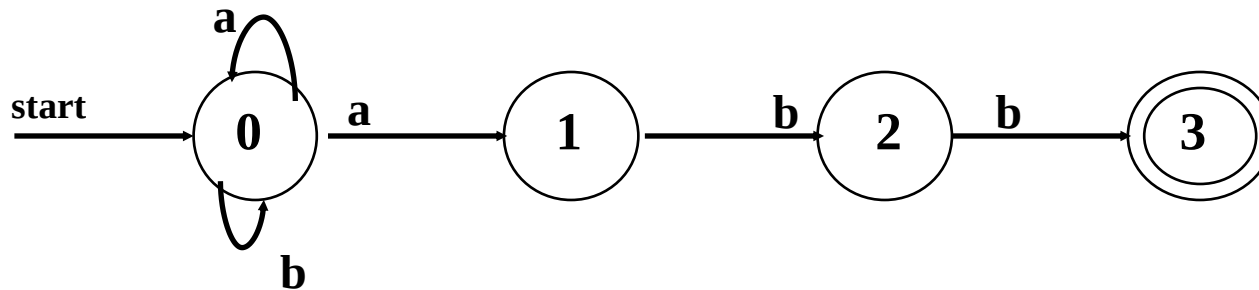
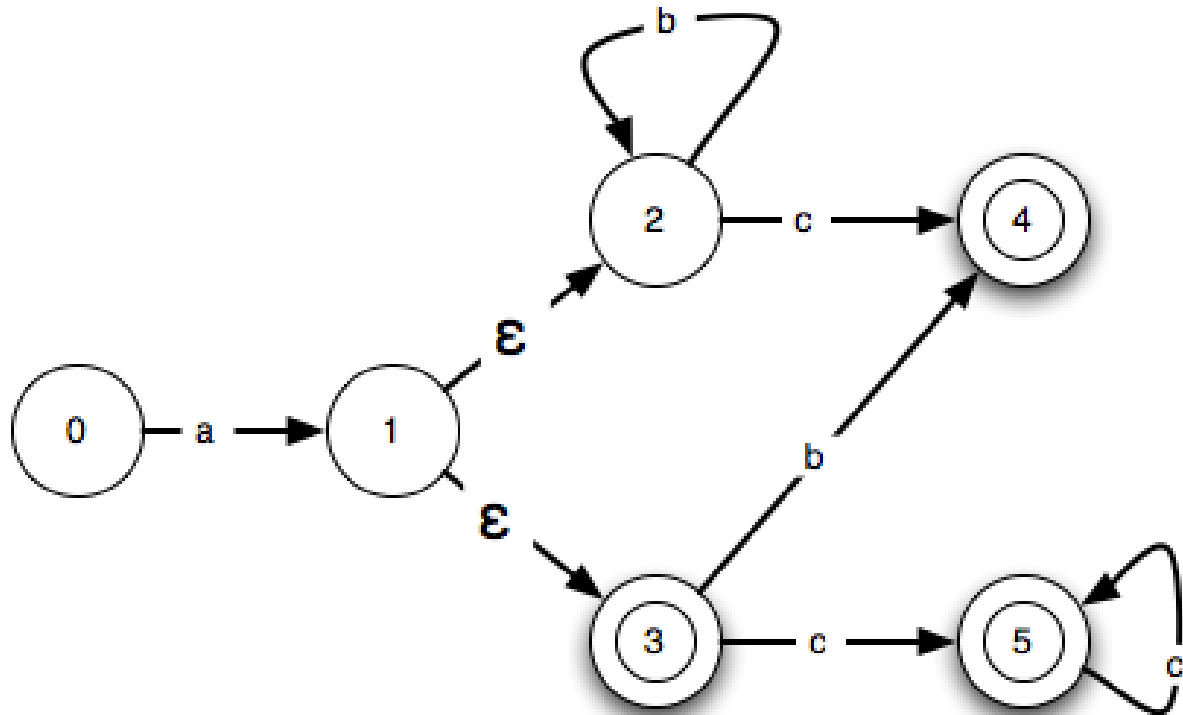


Figure 3.19: A nondeterministic finite automaton of expression $(a \mid b)^* abb$

3.6 Finite Automata

Find a transition diagram NFA that recognizes : $(a(b^*c)) \mid (a(b|c^*))$

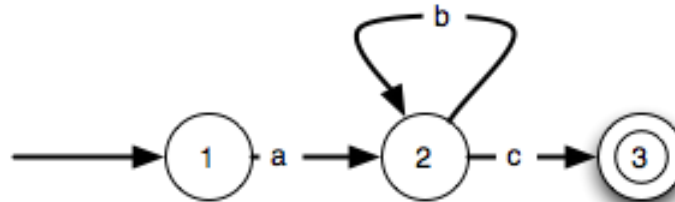
Solution:



3.6 Finite Automata

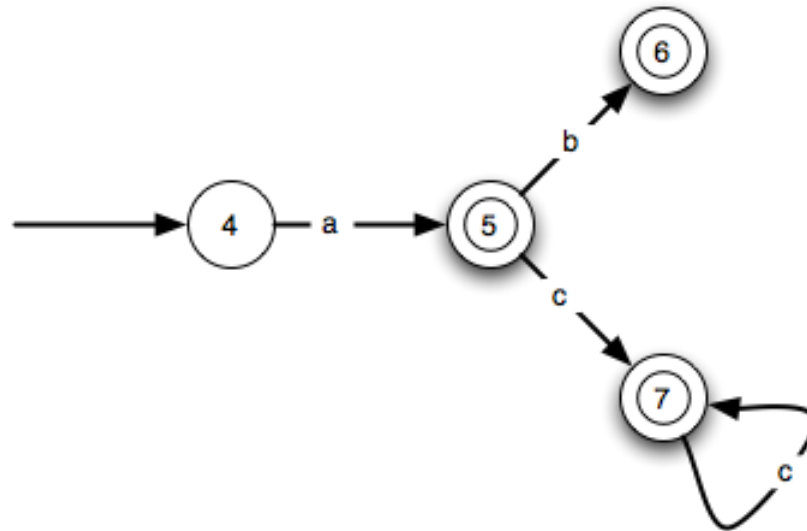
Find a transition diagram NFA that recognizes : $a(b^*c)$

Solution:



Find a transition diagram NFA that recognizes : $a(b|c^+)?$

Solution:

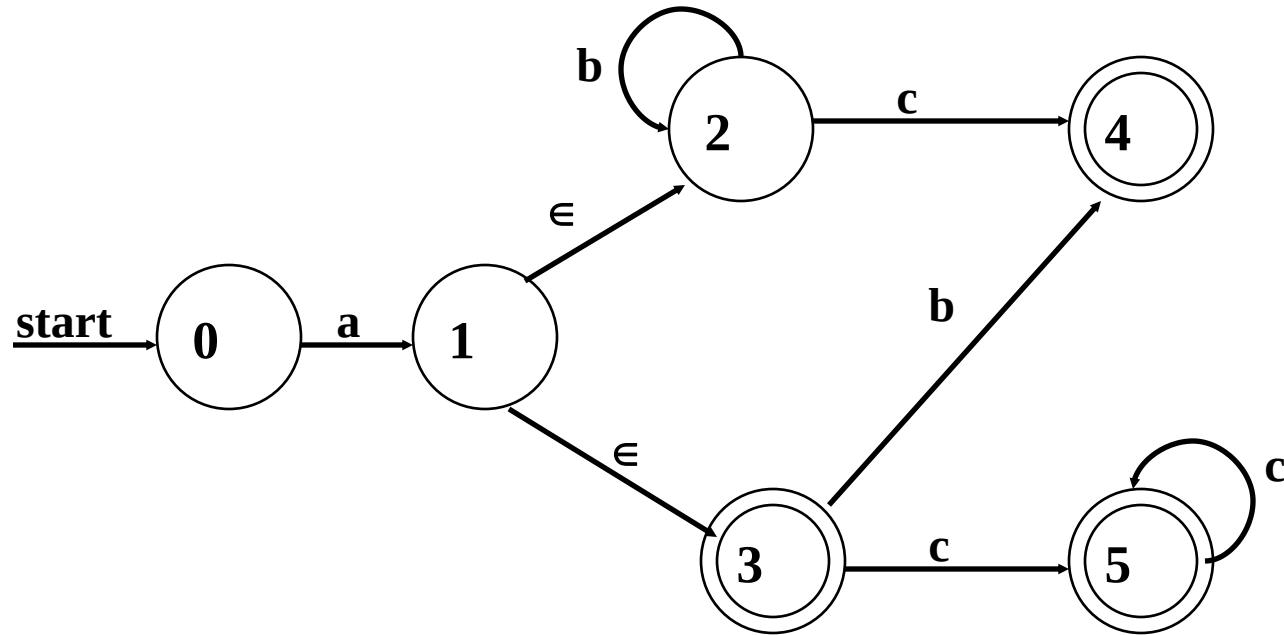


3.6 Finite Automata



Find a transition diagram NFA that recognizes : $(a(b^*c)) \mid (a(b \mid c^+)?)$

Solution:



3.6 Finite Automata

❖ A **DFA** is an NFA with the following restrictions:

- ✓ ϵ moves are not allowed
- ✓ For every state $s \in S$, there is one and only one path from s for every input symbol $a \in \Sigma$

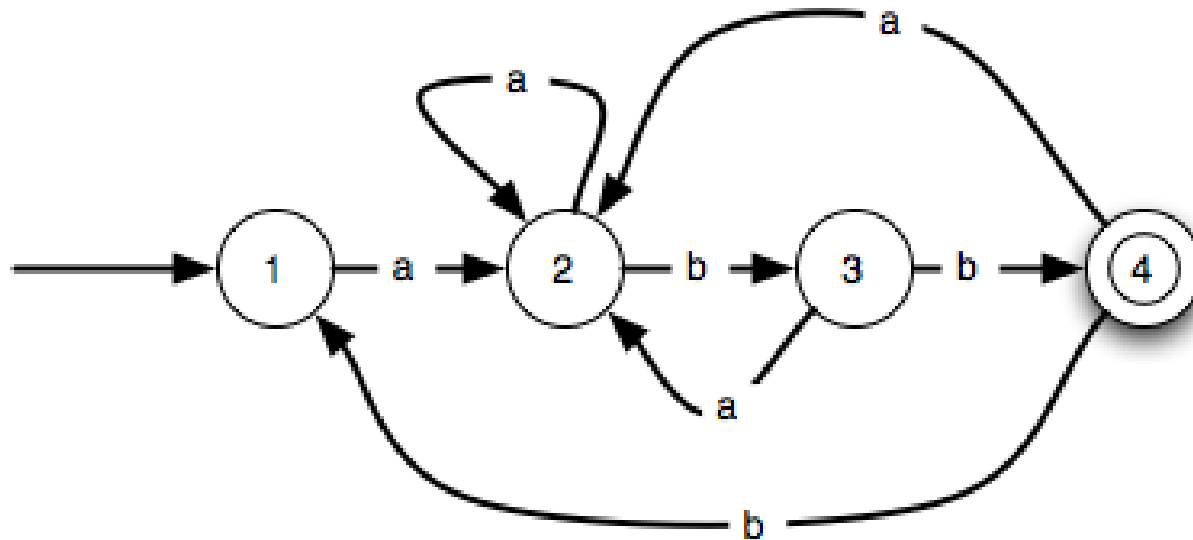


Figure 3.23: A deterministic finite automaton of expression $(a \mid b)^* abb$

3.6 Finite Automata



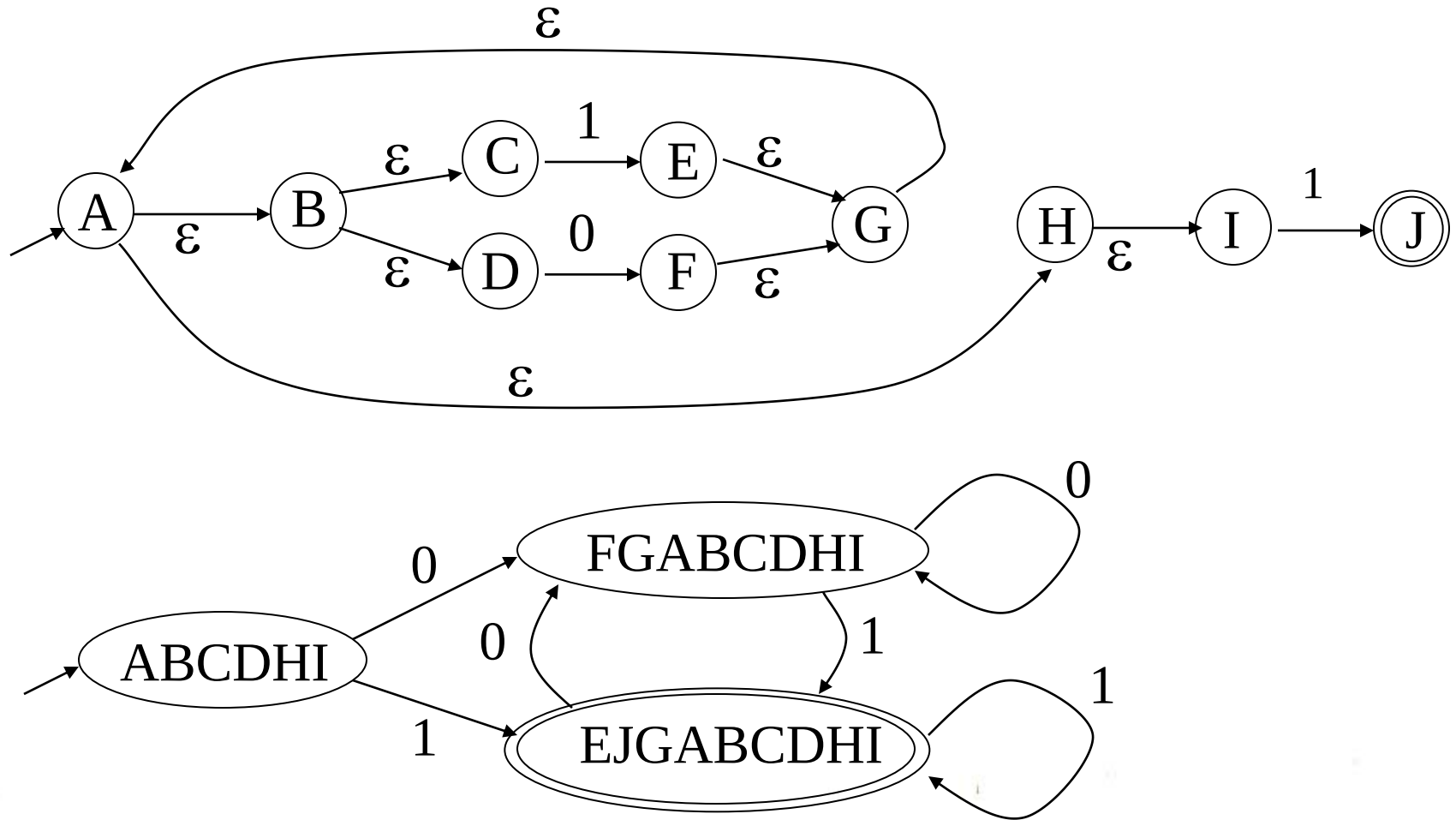
Conversion of an NFA into a DFA:

- Reasons to conversion
 - Avoiding ambiguity
- Simulate the NFA
- Each state of resulting DFA
 - = a non-empty subset of states of the NFA
- Start state
 - = the set of NFA states reachable through ϵ -moves from NFA start state
- Add a transition $S \xrightarrow{a} S'$ to DFA iff
 - S' is the set of NFA states reachable from the states in S after seeing the input a considering ϵ -moves as well

3.6 Finite Automata



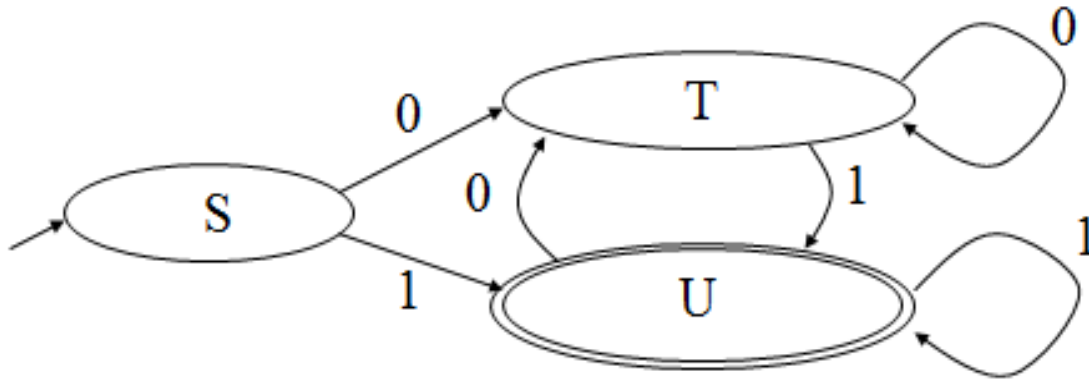
NFA -> DFA Example



3.6 Finite Automata

✚ NFA to DFA. Remark

- ✓ An NFA may be in many states at any time
- ✓ If there are N states, the NFA must be in some subset of those N states
- ✓ How many non-empty subsets are there?
 $2^N - 1 =$ finitely many, but exponentially many



	0	1
S	T	U
T	T	U
U	T	U

- ✓ NFA $\rightarrow$ DFA conversion is at the heart of tools such as flex or jflex
- ✓ But, DFAs can be huge
- ✓ In practice, flex-like tools trade off speed for space in the choice of NFA and DFA representations